

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Клиент-серверные приложения и сетевые технологии»

Пояснительная записка к курсовой работе

на тему:

«Разработка клиент-серверного приложения Toy Shelf для учета игрушек»

Выполнил: студент группы БИВТ-24-4

Кадыров Артур Рустамович

Подпись: _____

Проверили: Абросимов Никита Андреевич

Шелудяков Пётр Алексеевич

Москва, 2026

Оглавление

1. Предметная область	2
2. Постановка задачи	3
3. Описание архитектуры	4
4. Описание структуры данных	5
5. Описание серверной части	6
6. Описание клиентской части	7
7. Тестирование приложения	8
8. Заключение	9
9. Список литературы	10

1. Предметная область

Предметной областью курсовой работы является разработка персонального клиент-серверного веб-приложения для учета игрушек и пользователей. Приложение Toy Shelf ориентировано на демонстрацию базовых возможностей современного веб-стека: клиентского интерфейса, серверного API, валидации входных данных, обработки ошибок, логирования и управления доступом по категориям клиентов.

В рамках выбранной темы игрушка рассматривается как элемент каталога, а пользователь — как участник системы. Пользователи делятся на категории: гость, клиент и администратор. Гость может просматривать демонстрационные данные, клиент получает обычный пользовательский доступ, администратор может создавать новых пользователей через интерфейс.

Основная проблема предметной области состоит в необходимости централизованно получать данные о пользователях и демонстрационном каталоге, проверять корректность вводимых данных и ограничивать доступ к отдельным действиям в зависимости от категории пользователя.

2. Постановка задачи

Цель работы — разработать персональное клиент-серверное приложение Toy Shelf с использованием HTML, CSS, React, NestJS, REST API, LocalStorage и средств валидации данных.

Серверная часть должна предоставлять обязательные endpoint: GET /users для получения списка всех пользователей, GET /users/:id для получения пользователя по идентификатору и POST /users для создания нового пользователя.

Каждый пользователь содержит поля `id`, `name`, `email` и `age`. Поля `id`, `name` и `email` обязательны. Поле `age` является необязательным. Поле `email` должно быть валидным адресом электронной почты. Данные пользователей хранятся в памяти приложения и не сохраняются между перезапусками сервера.

Дополнительно реализованы endpoint `POST /auth/login` для демонстрационной авторизации и `GET /toys` для отображения каталога игрушек. Эти endpoint расширяют предметную область и позволяют показать управление доступом на клиентской стороне.

3. Описание архитектуры

Приложение построено по клиент-серверной архитектуре. Клиентская часть реализована на React и запускается отдельно через Vite. Серверная часть реализована на NestJS и предоставляет REST API в формате JSON. Обмен данными выполняется по HTTP с использованием Fetch API.

React-клиент отвечает за отображение интерфейса, отправку запросов, обработку пользовательского ввода и хранение демо-сессии в LocalStorage. NestJS-сервер отвечает за маршрутизацию запросов, валидацию DTO, вызов сервисов, формирование ответов и обработку исключений.

Слой хранения данных реализован в памяти серверного процесса. Такой подход соответствует условию задания: данные не сохраняются между перезапусками приложения. Для реальной эксплуатации этот слой можно заменить базой данных без изменения внешнего API.

Компонент	Назначение
React-клиент	Отображение интерфейса, работа с LocalStorage, отправка HTTP-запросов
NestJS-сервер	REST API, валидация, обработка ошибок и логирование
UsersModule	Работа с пользователями
AuthModule	Демонстрационная авторизация и категории доступа
ToysModule	Демонстрационный каталог игрушек

Таблица 1 — Компоненты архитектуры

4. Описание структуры данных

В проекте используется in-memory модель хранения. Основная сущность User описывает пользователя системы и содержит уникальный идентификатор, имя, email и необязательный возраст. Идентификатор создается сервером с помощью randomUUID.

Демонстрационная сущность Toy используется для отображения предметной области. Она содержит `id`, `title`, `category`, `price` и `available`. Каталог игрушек не является

обязательной частью API по заданию, но используется для раскрытия темы приложения и демонстрации клиентского интерфейса.

Так как данные хранятся в памяти, при каждом перезапуске сервера массив пользователей заново инициализируется начальными демонстрационными значениями. Это поведение специально сохранено, потому что оно прямо указано в задании.

Поле	Тип	Обязательность	Назначение
id	string	да	Уникальный идентификатор пользователя
name	string	да	Имя пользователя
email	string	да	Адрес электронной почты
age	number	нет	Возраст пользователя

Таблица 2 — Структура сущности User

5. Описание серверной части

Серверная часть разработана на NestJS. Главный модуль AppModule подключает UsersModule, AuthModule и ToysModule. Каждый модуль объединяет контроллеры и сервисы соответствующей области.

UsersController содержит обработчики GET /users, GET /users/:id и POST /users. Контроллер принимает HTTP-запросы, извлекает параметры и тело запроса, после чего передает работу в UsersService. UsersService хранит массив пользователей, выполняет поиск, создание пользователя и проверку уникальности email.

CreateUserDto содержит правила валидации данных: name должен быть непустой строкой, email должен соответствовать формату электронной почты, age является необязательным целым числом от 1 до 120. Для проверки используется библиотека class-validator, а преобразование типов выполняется через class-transformer.

В main.ts подключен глобальный ValidationPipe. Он выполняет проверку DTO, преобразует входные данные и запрещает лишние поля. Для обработки ошибок используется HttpExceptionFilter, который возвращает единый JSON-ответ со статусом, временем и деталями ошибки.

Логирование запросов реализовано через RequestLoggerMiddleware. Для каждого HTTP-запроса фиксируются метод, путь, код ответа и время выполнения. Это помогает анализировать работу приложения и быстрее находить ошибки при тестировании.

AuthService демонстрирует базовые принципы безопасности. Пароли демо-пользователей не хранятся как открытый текст: для сравнения используется bcrypt-хеш. В учебной версии возвращается демонстрационный token. В промышленном приложении его следует заменить на JWT или защищенные серверные сессии.

Endpoint	Метод	Назначение
/users	GET	Получение списка всех пользователей
/users/:id	GET	Получение пользователя по id
/users	POST	Создание пользователя
/auth/login	POST	Демонстрационная авторизация
/toys	GET	Получение каталога игрушек

Таблица 3 — Основные endpoint приложения

6. Описание клиентской части

Клиентская часть разработана на React. В файле App.jsx реализованы основные состояния приложения: текущая сессия, форма входа, список пользователей, список игрушек, форма создания пользователя и сообщение об ошибке.

HTML-разметка формируется через JSX. CSS-оформление вынесено в styles.css. Интерфейс содержит верхнюю панель с текущей ролью, форму входа, список игрушек, список пользователей и форму создания пользователя. На мобильных экранах сетка перестраивается в одну колонку.

Для обмена с сервером используется функция request из api.js. Она выполняет fetch-запрос, разбирает JSON-ответ и преобразует ошибки сервера в понятное сообщение для интерфейса.

LocalStorage используется для хранения демо-сессии пользователя. После входа данные сохраняются под ключом toy-shelf-session, а после выхода удаляются. При обновлении страницы приложение восстанавливает состояние входа из LocalStorage.

Управление доступом реализовано по категориям клиентов. Гость может войти без пароля и просматривать данные. Клиент входит с логином client и паролем client123. Администратор входит с логином admin и паролем admin123 и получает доступ к форме создания пользователя.

7. Тестирование приложения

Работоспособность приложения проверялась через интерфейс клиента и через прямые HTTP-запросы к серверу. Основные сценарии: получение списка пользователей, получение пользователя по id, создание пользователя, ошибка при некорректном email, ошибка при повторяющемся email и проверка исчезновения созданных данных после перезапуска сервера.

Пример успешного POST-запроса: POST http://localhost:3000/users с телом {"name":"Test User","email":"test@example.com","age":20}. В ответ сервер возвращает созданный

объект пользователя с автоматически сгенерированным id.

Пример ошибки валидации: POST `http://localhost:3000/users` с email, не соответствующим формату адреса электронной почты. В этом случае `ValidationPipe` отклоняет запрос, а фильтр исключений возвращает JSON-ответ с кодом ошибки и деталями.

8. Заключение

В результате выполнения курсовой работы разработано клиент-серверное приложение Toy Shelf. Серверная часть реализована на NestJS, клиентская часть — на React и Vite. Приложение демонстрирует работу REST API, HTML/JSX-разметки, CSS-оформления, LocalStorage, управления доступом по категориям пользователей и обработки HTTP-запросов.

В серверной части применены контроллеры, сервисы, провайдеры, DTO, декораторы NestJS, библиотека `class-validator`, обработка исключений и логирование запросов. В клиентской части реализована работа с API, сохранение сессии и условное отображение интерфейса в зависимости от роли.

Поставленная задача выполнена: обязательные endpoint `/users` реализованы, данные пользователей хранятся в памяти, email валидируется, обязательные поля проверяются, а работа приложения может быть продемонстрирована через браузер и HTTP-запросы.

9. Список литературы

[1] Официальная документация NestJS. URL: <https://docs.nestjs.com/>

[2] Официальная документация React. URL: <https://react.dev/>

[3] Документация MDN Web Docs по Fetch API. URL: https://developer.mozilla.org/docs/Web/API/Fetch_API

[4] Документация MDN Web Docs по LocalStorage. URL: <https://developer.mozilla.org/docs/Web/API/Window/localStorage>

[5] Документация `class-validator`. URL: <https://github.com/typestack/class-validator>

[6] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления.

[7] Исходный код приложения Toy Shelf: архив КР-6__БИБТ-24-4__Кадыров_Артур_Рустамович__Toy_Shelf_Source.zip.